

Relier textes, profils et synthèse contextualisée avec NaileR

Dernière case du parcours

Objectif de cette dernière case

Cette dernière case montre comment **NaileR** peut combiner deux sources d'information :

- les **verbatim**s des individus ;
- les **variables** qui décrivent les groupes.

L'objectif n'est pas seulement de produire une interprétation à partir de textes. Il est de montrer comment on peut construire deux artefacts complémentaires, puis les réunir dans une synthèse contextualisée.

Le cheminement est le suivant :

```
textes par groupe  
→ artefact textuel  
→ profil des groupes  
→ synthèse contextualisée
```

Charger NaileR

On commence par charger le package.

```
library(NaileR)
```

Dans cet exemple, les données sont simulées. Elles portent sur des attitudes vis-à-vis du covoiturage et de la mobilité partagée.

Créer un petit jeu de données synthétique

On crée trois groupes fictifs :

- des conducteurs pragmatiques ;
- des individus écologiquement flexibles ;
- des individus attachés à leur autonomie.

```

set.seed(123)

n <- 90

group <- sample(
  x = c("Pragmatic drivers", "Eco-flexible", "Autonomy first"),
  size = n,
  replace = TRUE,
  prob = c(0.35, 0.30, 0.35)
)

```

Générer des variables structurées

On crée ensuite quelques variables descriptives : lieu de résidence, habitude de transport et type d'emploi du temps.

```

residence <- sapply(group, function(g) {
  if (g == "Pragmatic drivers") {
    sample(c("Urban", "Peri-urban"), 1, prob = c(0.6, 0.4))
  } else if (g == "Eco-flexible") {
    sample(c("Urban", "Peri-urban", "Rural"), 1, prob = c(0.5, 0.3, 0.2))
  } else {
    sample(c("Rural", "Peri-urban"), 1, prob = c(0.7, 0.3))
  }
})

transport_habit <- sapply(group, function(g) {
  if (g == "Pragmatic drivers") {
    sample(c("Car", "Mixed"), 1, prob = c(0.75, 0.25))
  } else if (g == "Eco-flexible") {
    sample(c("Mixed", "Public transport", "Bike"), 1, prob = c(0.45, 0.30, 0.25))
  } else {
    sample(c("Car", "Mixed"), 1, prob = c(0.85, 0.15))
  }
})

schedule_type <- sapply(group, function(g) {
  if (g == "Pragmatic drivers") {
    sample(c("Variable", "Fixed"), 1, prob = c(0.7, 0.3))
  } else if (g == "Eco-flexible") {
    sample(c("Fixed", "Variable"), 1, prob = c(0.65, 0.35))
  } else {
    sample(c("Variable", "Fixed"), 1, prob = c(0.75, 0.25))
  }
})

```

Générer une variable textuelle

Chaque individu reçoit maintenant un court commentaire. Le texte est cohérent avec son groupe.

```
make_text <- function(g, res, habit, sched) {  
  if (g == "Pragmatic drivers") {  
    base <- sample(c(  
      "Carpooling can be useful, but in daily life time constraints make it difficult.",  
      "I see environmental benefits, but I need something practical and easy to organize.",  
      "I could share some journeys, but only when the schedule is simple.",  
      "The main issue is efficiency: I do not want to lose time in everyday travel."  
    ), 1)  
  
    add <- sample(c(  
      "My schedule changes often, so I cannot always adapt to others.",  
      "If the trip is planned in advance, I am more open to sharing.",  
      "Public transport is available, but it is not always the fastest option.",  
      "I mainly think in terms of convenience and feasibility."  
    ), 1)  
  
  } else if (g == "Eco-flexible") {  
    base <- sample(c(  
      "Reducing emissions matters to me, so I am generally favorable to shared mobility.",  
      "I try to limit solo car use whenever I can.",  
      "Carpooling makes sense environmentally and socially.",  
      "I am quite open to alternatives to driving alone."  
    ), 1)  
  
    add <- sample(c(  
      "If the organization is reasonable, I am willing to adapt.",  
      "I already combine several transport options depending on the situation.",  
      "For regular trips, collective solutions seem realistic.",  
      "The environmental argument is important in my decisions."  
    ), 1)  
  
  } else {  
    base <- sample(c(  
      "I need autonomy in my journeys and I do not want to depend on other people's schedules.",  
      "My car gives me freedom, especially when trips are irregular.",  
      "In theory carpooling is interesting, but in practice I need independence.",  
      "I prefer keeping control over departure times and route changes."  
    ), 1)  
  
    add <- sample(c(  
      "Living outside dense urban areas makes flexibility essential.",  
      "Unexpected events make shared travel complicated.",  
      "My daily organization depends on being able to leave when I want.",  
      "I am not against the idea, but autonomy comes first."  
    ), 1)  
  }  
}
```

```

context <- paste(
  "Residence:", res, "|",
  "Habit:", habit, "|",
  "Schedule:", sched, "."
)

paste(base, add, context)
}

text_answer <- mapply(
  FUN = make_text,
  g = group,
  res = residence,
  habit = transport_habit,
  sched = schedule_type,
  SIMPLIFY = TRUE
)

```

Construire le tableau final

Le tableau contient une variable de groupe, une variable textuelle et plusieurs variables descriptives.

```

dta_text <- data.frame(
  Group = factor(group),
  Text = text_answer,
  Residence = factor(residence),
  TransportHabit = factor(transport_habit),
  ScheduleType = factor(schedule_type),
  stringsAsFactors = FALSE
)

str(dta_text)
table(dta_text$Group)

```

La variable **Group** est la variable qui définit les groupes à comparer. La variable **Text** contient les commentaires libres.

Première étape : `nail_textual()`

La fonction `nail_textual()` prépare un prompt à partir des textes regroupés par modalité de **Group**.

Ici, on met `generate = FALSE` pour inspecter le prompt sans appeler de modèle de langage.

```

res_textual_prompt <- nail_textual(
  dataset = dta_text,
  num.var = 1, # Group
  num.text = 2, # Text

```

```

  generate = FALSE
)

names(res_textual_prompt)
attr(res_textual_prompt, "textual_data_summary")

cat(res_textual_prompt[[1]])

```

À ce stade, on ne demande pas encore au modèle de produire une interprétation. On vérifie d'abord ce qu'on va lui transmettre.

Génération optionnelle avec `nail_textual()`

Si un modèle local est disponible, on peut générer une réponse.

```

# res_textual <- nail_textual(
#   dataset = dta_text,
#   num.var = 1,
#   num.text = 2,
#   model = "llama3",
#   generate = TRUE
# )

```

Deuxième étape : `nail_textual_prep()`

La fonction `nail_textual_prep()` prépare un artefact textuel par groupe.

Elle peut repérer les thèmes principaux, des verbatims représentatifs et des expressions notables.

```

res_textual_prep <- nail_textual_prep(
  dataset = dta_text,
  num.var = 1,
  num.text = 2,
  model = "llama3",
  generate = TRUE
)

names(res_textual_prep)

```

On peut regarder le résultat pour un groupe.

```

res_textual_prep[[1]]$parsed
res_textual_prep[[1]]$selected_verbatims
res_textual_prep[[1]]$notable_expressions

```

Cet objet est un premier artefact. Il résume ce que les textes disent de chaque groupe.

Troisième étape : `nail_group_profile_prep()`

Les textes ne sont pas la seule information disponible.

La fonction `nail_group_profile_prep()` prépare un profil des groupes à partir des variables du tableau.

```
res_group_profile <- nail_group_profile_prep(  
  dataset = dta_text,  
  num.var = 1,  
  model = "llama3",  
  generate = TRUE  
)  
  
names(res_group_profile)  
res_group_profile[[1]]$parsed
```

On obtient ici un second artefact. Il décrit les groupes à partir des variables comme `Residence`, `TransportHabit` ou `ScheduleType`.

Quatrième étape : `nail_textual_contextualized()`

La dernière fonction combine les deux artefacts :

- l'artefact textuel ;
- l'artefact structuré.

L'objectif est de produire une synthèse contextualisée.

```
res_contextualized_prompt <- nail_textual_contextualized(  
  group_profile_prep = res_group_profile,  
  textual_prep = res_textual_prep,  
  interpretation_mode = "comparative",  
  generate = FALSE  
)  
  
cat(res_contextualized_prompt)
```

Avec `generate = FALSE`, on inspecte le prompt final. C'est utile pour vérifier que la synthèse repose bien sur les deux sources d'information.

Génération finale optionnelle

Si le modèle local est disponible, on peut demander la synthèse finale.

```
# res_contextualized <- nail_textual_contextualized(
#   group_profile_prep = res_group_profile,
#   textual_prep = res_textual_prep,
#   interpretation_mode = "comparative",
#   model = "llama3",
#   generate = TRUE
# )
#
# res_contextualized$response
```

Synthèse finale : les artefacts produits

Cette étape ne produit pas directement une interprétation à partir des textes bruts. Elle construit progressivement plusieurs artefacts. Chaque artefact a un rôle précis dans la préparation du prompt final.

```
textes bruts
→ prompt textuel initial
→ artefact textuel par groupe
→ artefact structuré par groupe
→ prompt contextualisé
→ interprétation finale éventuelle
```

1. Le prompt textuel initial

Le premier objet est créé par `nail_textual()`.

Il sert à transformer les textes regroupés par modalité de `Group` en une requête interprétative.

Ce n'est pas encore une synthèse contextualisée. C'est une première mise en forme des textes pour demander au modèle de langage de décrire les différences entre groupes.

```
res_textual_prompt <- nail_textual(
  dataset = dta_text,
  num.var = 1, # Group
  num.text = 2, # Text
  generate = FALSE
)

cat(res_textual_prompt[[1]])
```

On peut aussi inspecter le résumé préparé par la fonction.

```
attr(res_textual_prompt, "textual_data_summary")
```

Cet objet permet de vérifier la requête avant toute génération.

2. L'artefact textuel préparé

Le deuxième objet est créé par `nail_textual_prep()`.

C'est l'artefact textuel central du workflow. Il ne se contente pas de créer un prompt : il prépare un profil textuel structuré pour chaque groupe.

```
res_textual_prep <- nail_textual_prep(  
  dataset = dta_text,  
  num.var = 1,  
  num.text = 2,  
  model = "llama3",  
  generate = TRUE  
)
```

Pour chaque groupe, l'objet peut contenir plusieurs éléments utiles.

```
names(res_textual_prep[[1]])
```

Le champ le plus important est généralement `parsed`.

```
res_textual_prep[[1]]$parsed
```

Il contient une synthèse structurée du matériau textuel du groupe :

```
profil textuel central  
thèmes principaux  
préoccupations dominantes  
tonalité ou position générale  
cohérence interne du groupe  
résumé injectable dans un prompt  
indices de verbatims centraux  
indices de verbatims de tension
```

L'artefact peut aussi contenir des verbatims sélectionnés.

```
res_textual_prep[[1]]$selected_verbatims
```

Ces verbatims permettent de distinguer deux choses :

- des verbatims centraux, représentatifs du profil ;
- des verbatims de “tension”, qui introduisent des contradictions internes.

Enfin, l'objet peut conserver des expressions singulières repérées dans les textes.

```
res_textual_prep[[1]]$notable_expressions
```

Ces expressions servent à rapprocher l'interprétation finale des mots employés par les individus.

3. L'artefact structuré des groupes

Le troisième objet est créé par `nail_group_profile_prep()`.

Il décrit les mêmes groupes, mais à partir des variables du tableau.

Dans cet exemple, il s'agit de variables comme `Residence`, `TransportHabit` et `ScheduleType`.

```
res_group_profile <- nail_group_profile_prep(  
  dataset = dta_text,  
  num.var = 1,  
  model = "llama3",  
  generate = TRUE  
)
```

On inspecte ensuite le profil préparé pour un groupe.

```
res_group_profile[[1]]$parsed
```

Cet artefact résume les groupes à partir des variables du tableau.

Il peut contenir notamment :

```
profil général du groupe  
traits quantitatifs distinctifs si présents  
traits catégoriels distinctifs  
marqueurs caractéristiques  
force ou netteté du profil  
résumé injectable dans un prompt
```

Dans ce workflow, cet artefact joue le rôle de contrepoint structuré à l'artefact textuel.

Les textes disent ce que les individus expriment. Les variables décrivent qui ils sont.

4. Le prompt contextualisé final

Le quatrième objet est créé par `nail_textual_contextualized()`.

Il combine les deux artefacts précédents :

- l'artefact textuel préparé avec `nail_textual_prep()` ;
- l'artefact structuré préparé avec `nail_group_profile_prep()`.

```
res_contextualized_prompt <- nail_textual_contextualized(  
  group_profile_prep = res_group_profile,  
  textual_prep = res_textual_prep,  
  interpretation_mode = "comparative",  
  generate = FALSE  
)  
  
cat(res_contextualized_prompt)
```

Avec `generate = FALSE`, on obtient le prompt final sans appeler le modèle.

Ce prompt final contient les informations nécessaires pour produire une interprétation contextualisée :

```
profil structuré des groupes
profil textuel des groupes
verbatim représentatifs
expressions notables
consignes d'interprétation
format attendu de la réponse
```

Selon l'objet retourné, certains éléments peuvent aussi être inspectés comme attributs.

```
attr(res_contextualized_prompt, "group_profile_summary")
attr(res_contextualized_prompt, "textual_group_summary")
attr(res_contextualized_prompt, "representative_verbatims")
attr(res_contextualized_prompt, "notable_expressions")
```

Cette inspection est importante : elle permet de vérifier que le prompt final repose bien sur des éléments préparés et non sur une demande vague adressée au modèle.

5. L'interprétation finale optionnelle

La dernière étape consiste à envoyer le prompt contextualisé au modèle de langage.

Cette étape est optionnelle dans le tutoriel, car elle dépend de la disponibilité d'un modèle local.

```
# res_contextualized <- nail_textual_contextualized(
#   group_profile_prep = res_group_profile,
#   textual_prep = res_textual_prep,
#   interpretation_mode = "comparative",
#   model = "llama3",
#   generate = TRUE
# )
#
# res_contextualized$response
```

Le point essentiel est que la réponse finale n'est pas produite à partir de données brutes.

Elle est produite à partir d'un prompt qui rassemble plusieurs artefacts déjà construits et vérifiables.

À retenir

Cette dernière case montre la logique complète du workflow textuel avec NaileR.

On part de commentaires libres, mais on ne les interprète pas isolément.

On construit d'abord un artefact textuel. Puis on construit un artefact structuré à partir des variables du tableau. Enfin, on combine les deux dans une synthèse contextualisée.

Le modèle n'interprète donc pas seul. Il travaille à partir d'artefacts préparés, visibles et vérifiables.

La chaîne complète peut se résumer ainsi :

```
verbatim  
→ nail_textual_prep()  
→ artefact textuel  
→ nail_group_profile_prep()  
→ artefact structuré  
→ nail_textual_contextualized()  
→ prompt contextualisé  
→ interprétation assistée
```

L'intérêt de cette démarche est de garder le contrôle sur l'interprétation : on peut vérifier les textes sélectionnés, les profils construits, les résumés injectés dans le prompt et la requête finale adressée au modèle.